

TECHNISCHE BEITRÄGE & ERFAHRUNGEN

Angular Upgrade & Paket-Kompatibilität

Ich habe alle Angular-Upgrades zwischen Version 14 und 18 erfolgreich durchgeführt. Im Rahmen dieser Upgrades habe ich sämtliche genutzten Bibliotheken und Abhängigkeiten überprüft und mit den neuen Versionen kompatibel gemacht. Zur Identifikation von veraltetem oder inkompatiblem Code habe ich individuelle Linting-Regeln entwickelt und implementiert.

Circular Dependencies – Erkennung & Vermeidung

Ich habe bestehende Circular-Dependency-Probleme identifiziert und die betroffenen Komponenten-, Service- und Modellstrukturen refaktoriert. Zusätzlich habe ich ein System zur frühzeitigen Erkennung solcher Abhängigkeitsfehler entwickelt – inklusive eigener Skripte, die sowohl in Pre-Commit-Hooks als auch in CI-Pipelines integriert sind. Entwickler*innen erhalten dadurch bereits beim Commit direkte Rückmeldungen über potenzielle Fehler.

Codequalität & Architekturregeln mit ESLint

Durch individuell angepasste ESLint-Regeln habe ich architekturspezifische Vorgaben im Team durchgesetzt. Beispielsweise werden versehentlich im Code verbliebene console.log-Statements oder cy.only-Markierungen in Cypress automatisch erkannt und vor dem Commit blockiert. Dies führt zu höherer Codequalität und vermeidet typische Fehlerquellen.

Testing & Testabdeckung

Ich achte darauf, dass jede Änderung und jede neue Funktionalität mit automatisierten Tests abgesichert wird, um langfristige Stabilität zu gewährleisten. Ich habe Erfahrung mit Cypress und bin in der Lage, neue Features direkt in bestehende Teststrukturen zu integrieren. Ebenso bin ich flexibel und kann mit anderen Testing-Frameworks arbeiten.

Beispiel-Feature: Activities-Seite

In meinem letzten Projekt war ich verantwortlich für die Konzeption und Entwicklung der "Activities"-Seite, auf der benutzerspezifische Aktivitäten wie Dokumentenversand, E-Mail-Kommunikation über die Plattform und Kalenderereignisse übersichtlich dargestellt werden.

Ein zentrales Element war die Möglichkeit, individuelle Aktivitäten über einen Dialog dynamisch zu erstellen – inklusive:

- Rich-Text-Editor zur Texteingabe,
- Datei-Uploads,
- und Benutzer-Tagging.

Diese custom Aktivitäten werden in einer generischen, wiederverwendbaren Tabellenkomponente dargestellt. Die Tabelle bietet folgende Funktionen:

- Filterbare Spalten zur schnellen Datenselektion,
- Dynamisches Hinzufügen neuer Spalten mit minimalem Entwicklungsaufwand,
- Export-Funktion, bei der gezielt auswählbare Spalten exportiert werden können,
- und Pagination, bei der jeweils nur der benötigte Datensatz vom Backend abgefragt wird (Lazy Loading).

Die gesamte Komponente wurde systemübergreifend eingebunden und folgt den etablierten Architekturprinzipien.